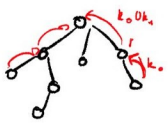
Exercice 1. *Communications collectives.*

a) Expliquez en quoi consiste la concentration dans un système distribué. Quelle commande peut-on utiliser pour effectuer une concentration avec `MPI` ? Écrivez un programme C avec la librairie `MPI` qui concentre la valeur d'un entier i vers l'entité d'identifiant 0 en utilisant cette commande. La valeur de i sera choisie aléatoirement sur chaque entité de calcul.

Dans un système distribué, la concentration est l'action d'envoyer, pour chaque entité de calcul i, un message m_i à une entité de calcul r. Nous pouvons utiliser la commande `MPI_Gather`




```c
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"

int main() {
    int id, nb, i, *buffer;

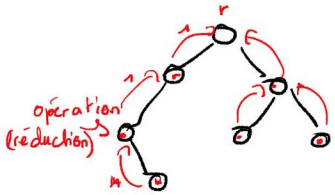
    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &nb);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    srand(time(NULL)+id);
    i = (rand() % 100)+1      // nb entre 1 et 100

    buffer = malloc((nb-1) * sizeof(int));

    MPI_Gather(&i, 1, MPI_INT, buffer, 1, MPI_INT, 0, MPI_COMM_WORLD);

    MPI_Finalize();
    return EXIT_SUCCESS;
}
```

b) Expliquez en quoi consiste la réduction dans un système distribué. Quelle commande peut-on utiliser pour effectuer une réduction avec `MPI` ? Écrivez un programme C avec la librairie `MPI` qui réduit la valeur d'un entier i vers l'entité d'identifiant 0 en utilisant cette commande et l'opération `MPI_PROD`. La valeur de i sera choisie aléatoirement sur chaque entité de calcul.



Dans une réduction, chaque entité de calcul (sauf la racine) reçoit un message de chacun de ses fils, pour chacune de ces réceptions, il effectue une opération avec son propre message. Après réception de nombre de chaque fils, il envoie à son père le résultat de cette succession de réductions.
On utilise la commande `MPI_reduce`:

```c
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"
int main() {
    int id, nb, i, buffer;

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &nb);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    srand(time(NULL)+id);
    i = (rand() % 100)+1      // nb entre 1 et 100

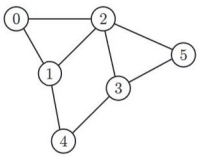
    MPI_reduce(&i, &buffer, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);

    MPI_Finalize();
    return EXIT_SUCCESS;
}
```

c) Expliquez en quoi consiste la barrière de synchronisation dans un système distribué. Quelle commande peut-on utiliser pour effectuer une barrière de synchronisation avec `MPI` ? Quels arguments cette commande prend-elle ?

La barrière de synchronisation va bloquer les entités de calcul tant que toutes ne sont pas bloquées.
On utilise la commande `MPI_Barrier(MPI_Comm_World)` pour effectuer une barrière de synchronisation, où `MPI_COMM_WORLD` est le communicateur des entités concernées.

Exercice 2. On considère un système distribué dont le graphe est le suivant :



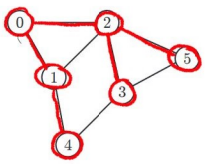
Sur ce dessin, les nombres sont les identifiants des entités de calcul.

a) Donnez la définition d'un arbre de parcours en largeur (ou BFS) de racine r.

Un BFS de racine r est un arbre recouvrant T de G tq ∀ sommet v de G :

$$d_T(r,v) = d_G(r,v)$$

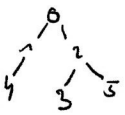
b) Dans le système distribué décrit ci-dessus, donner un arbre de parcours en largeur de racine 0. Combien y a t-il de tels arbres ? Pourquoi ?



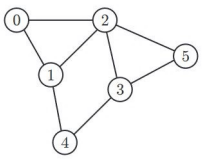
Il n'y a qu'un seul BFS dans cet exemple. En effet, ∀ sommet s, 3! chemin minimal entre r et s.

c) Écrivez l'algorithme de Bellman-Ford en pseudo-code. Voir cours

d) Appliquez l'algorithme de Bellman-Ford au système distribué décrit ci-dessus, en prenant comme racine $r = 0$. Pour cela, tracez le déroulement de cet algorithme sur chacune des 6 entités de calcul séparément.



L'algorithme Bellman-Ford appliqué à notre système donne la trace suivante :



0	1	2	3	4	5
Fils = ∅ Fin = 0 Envoie 0 à 1 (Donnée) Envoie 0 à 2 (Donnée) c ← 4	Fils = ∅ Fin = 0 Reçoit d de 0 (donnée) L ← 1 [d=0] père ← 0 c ← 4 Envoie 1 à 2 (donnée) Envoie 1 à 4 (donnée) Reçoit T1 de 2 (donnée) Envoie T1 [R=1] à 2 (AR) c ← 3 ...	Fils = ∅ Fin = 0 Reçoit d de 0 (donnée) L ← 1 [d=0] père ← 0 c ← 6 Envoie 1 à 1 (donnée) Envoie 1 à 3 (donnée) Envoie 1 à 5 (donnée) Reçoit T1 de 1 (donnée) Envoie T1 [R=1] à 1 (AR) c ← 5 ...	Fils = ∅ Fin = 0 Reçoit d de 2 (donnée) L ← 2 [d=1] père ← 2 c ← 4 Envoie 2 à 4 (donnée) Envoie 2 à 5 (donnée) Reçoit T1 de 4 (donnée) Envoie T1 [R=2] à 4 (AR) c ← 3 ...	Fils = ∅ Fin = 0 Reçoit d de 1 (donnée) L ← 2 [d=1] père ← 1 c ← 2 Envoie 2 à 3 (donnée) Reçoit T1 de 3 (donnée) Envoie T1 [R=2] à 3 (AR) c ← 1 ...	Fils = ∅ Fin = 0 Reçoit d de 2 (donnée) L ← 2 [d=1] père ← 2 c ← 3 Envoie 2 à 3 (donnée) ...

À FINIR

Exercice 3. On dispose du communicateur `MPI_COMM_WORLD` contenant n entités de calcul d'identifiant 0, 1, ..., $n-1$.

a) Comment devez-vous procéder pour diffuser un message (disons un entier x préalablement initialisé) à partir de l'entité de calcul d'indentifiant 1 vers toutes les autres entités de calcul d'identifiant impair (et uniquement vers celles-là) en utilisant la commande `MPI_Bcast` ? Écrivez la portion de programme qui vous permet de faire cela.

Afin de diffuser un message de la sorte, nous pouvons implémenter la portion de programme suivante:

```
int message, couleur;   (id: identifiant)
MPI_Comm comm;
if (id %2) couleur = 1;
else couleur = MPI_UNDEFINED;
MPI_Comm_split (MPI_COMM_WORLD, couleur, (id - id%2)/2, &comm)
if (id == 1) message = 42;
MPI_Bcast (&message, 1, MPI_INT, 0, comm);
```

0 1 2 3 4 5



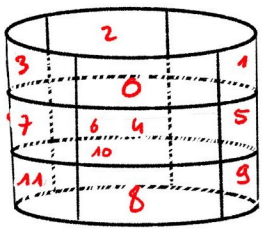
b) Supposons que $n = 12$. Nous voulons créer un communicateur `comm` de topologie cartésienne 2-dimensionnelle dans lequel il y a 4 entités de calcul dans la première dimension et 3 entités de calcul dans la deuxième dimension. La première dimension de ce communicateur doit être périodique et la deuxième dimension doit être non-périodique. Quelles instructions devez-vous utiliser pour créer le communicateur `comm` à partir du communicateur `MPI_COMM_WORLD` ?

0 1 ... 11

L'instruction que nous devons utiliser dans ce cas est :

```
MPI_Comm comm;
MPI_Cart_create(MPI_COMM_WORLD, 2, [4,3], [1,0], 1 &comm);
```

c) Dans le communicateur `comm` défini à la question précédente, donnez le nombre de voisins de chaque entité de calcul.



l'entité 0 a 3 voisins
l'entité 1 a 3 voisins
l'entité 2 a 3 voisins
l'entité 3 a 3 voisins
l'entité 4 a 4 voisins
l'entité 5 a 4 voisins
l'entité 6 a 4 voisins
l'entité 7 a 4 voisins
l'entité 8 a 3 voisins
l'entité 9 a 3 voisins
l'entité 10 a 3 voisins
l'entité 11 a 3 voisins

d) Supposons qu'on dispose du communicateur `comm` défini à la question b). Supposons aussi que chaque entité de calcul possède un entier p. Écrire une portion de programme (en C avec la librairie `MPI`) qui effectue les étapes suivantes :

i. Affecte une valeur choisie aléatoirement entre 0 et 99 à l'entier p,
ii. Remplace la valeur de p de l'entité de calcul courante par le maximum des valeurs de p des entités de calcul voisines dans le communicateur `comm`.

Voici une portion de programme effectuant ce qui est demandé:

```
srand (time(NULL) + id)
p = rand() % 100;
int prec;
int voisins[4];
MPI_Cart_shift(comm, 1, 1, voisins, voisins+1);
MPI_Cart_shift(comm, 2, 1, voisins+2, voisins+3);
for(int i=0; i<4; i++) {
  if (voisins+i != MPI_PROC_NULL)
      MPI_Send(&p, 1, MPI_INT, voisins+i, 0, comm);
}
for(int i=0; i<4; i++) {
   if (voisins+i != NULL){
      MPI_Recv(&prec, 1, MPI_INT, voisins+i, 0, comm, NULL)
      if(prec>p) p = prec;
   }
}
```

e) Soit i l'entité ayant tiré le plus grand nombre p. Appelons ce p pmax. Lors de la 1er itération, la variable p de chaque voisin de i est pmax. À l'itération suivante, p des voisins des voisins de i vaut pmax

etc... Ainsi, p converge vers pmax.